Использование различных техник тестирования

ПОЗИТИВНОЕ ТЕСТИРОВАНИЕ

тестирование, при котором используются только валидные данные и выполняются только валидные действия

НЕГАТИВНОЕ ТЕСТИРОВАНИЕ

тестирование с использованием невалидных данных и действий, направленное на получение адекватной реакции системы

Позитивные и негативные тест-кейсы

При планировании тестирования предлагается ответить на следующие вопросы относительно тестируемого объекта:

- Что перед вами?
- Кому и зачем оно нужно (и насколько это важно)?
- Как оно обычно используется?
- Как оно может сломаться, т.е. начать работать неверно?

Пример из книги Куликова.

1. Тестируем приложение **Конвертор файлов**, которое служит для автоматического приведения кодировок текстовых документов к одной.

2. Выберем для исследования первый параметр командной строки — SOURCE_DIR — имя каталога, в котором приложение ищет файлы, подлежащие конвертации.

3. Читаем спецификацию

Пользовательские требования

- Также см. диаграмму вариантов использования.
- ПТ-1: Запуск и остановка приложения.
  - ПТ-1.1: Запуск приложения производится из консоли командой «php converter.php SOURCE_DIR DESTINATION_DIR [LOG_FILE_NAME]» (описание параметров приведено в разделе ДС-2.1, реакция на ошибки при указании параметров приведена в разделах ДС-2.2, ДС-2.3, ДС-2.4).
  - ПТ-1.2: Остановка приложения производится выполнением команды Ctrl+C в окне консоли, из которого было запущено приложение.

Детальные спецификации ДС

ДС-2: Параметры командной строки

ДС-2.1: При запуске приложения оно получает из командной строки три параметра:

SOURCE_DIR — обязательный параметр, определяет путь к каталогу с файлами, которые необходимо обработать;

DESTINATION_DIR — обязательный параметр, определяет путь к каталогу, в который необходимо поместить обработанные файлы (этот каталог не может находиться внутри каталога SOURCE_DIR или в его подкаталогах (см. БП-1.1 и БП-1.2));

LOG_FILE_NAME — необязательный параметр, определяет полное имя лог-файла (по умолчанию лог-файл с именем «converter.log» размещается по тому же пути, по которому находится файл скрипта converter.php);

Что перед нами? Путь к каталогу. Казалось бы, просто, но стоит вспомнить, что наше приложение должно работать как минимум под управлением Windows и Linux, что приводит к необходимости освежить в памяти принципы работы файловых систем в этих ОС. А ещё может понадобиться работа с сетью.

Кому и зачем оно нужно (и насколько это важно)? Конечные пользователи не занимаются конфигурированием приложения, т.е. этот параметр нужен администратору (предположительно, это человек квалифицированный и не делает явных глупостей, но из его же квалификации вытекает возможность придумать такие варианты использования, до которых не додумается рядовой пользователь). Важность параметра критическая, т.к. при каких-то проблемах с ним есть риск полной потери работоспособности приложения.

Как оно обычно используется? Здесь понадобится понимание принципов работы файловых систем.

Корректное имя существующего каталога:

Windows:

- X:\dir
- “X:\dir with spaces”
- .\dir
- ..\dir
- \\host\dir
- Всё вышеперечисленное с “\” в конце пути.
- X:\

Linux:

- /dir
- “/dir with spaces”
- host:/dir
- smb://host/dir
- ./dir
- ../dir
- Всё вышеперечисленное с “/” в конце пути.
- /

получили чек-лист для позитивного тестирования

Как оно может сломаться, т.е. начать работать неверно? Итак, какие проблемы с именем каталога-источника (и самим каталогом источником) могут помешать нашему приложению?

- Указанный путь не является корректным именем каталога:
  - Пустое значение (“”).
  - Слишком длинное имя:
    - Для Windows: более 256 байт. (Важно! Путь к каталогу длиной в 256 байт допустим, но надо учесть ограничение на полное имя файла, т.к. его превышение может быть достигнуто естественным образом, что приведёт к возникновению сбоя.)
    - Для Linux: более 4096 байт.
  - Недопустимые символы, например: ? < > \ * | " \0.
  - Недопустимые комбинации допустимых символов, например: “....\dir”.
- Каталог не существует:
  - На локальном диске.
  - В сети.
- Каталог существует, но к нему нет прав доступа.
- Доступ к каталогу утерян после запуска приложения:
  - Каталог удалён или переименован.
  - Изменены права доступа.
  - Потеря соединения с удалённым компьютером.
- Использование зарезервированного имени:
  - Для Windows: com1-com9, lpt1-lpt9, con, nul, prn.
  - Для Linux: “..”.
- Проблемы с кодировками, например: имя указано верно, но не в той кодировке.

Негативных тест-кейсов оказывается намного больше, чем позитивных.

Возникает два вопроса:

- ✓ Все ли эти варианты надо проверить?
- ✓ Не упустили ли мы что-то важное?

- ✓ Все ли варианты надо проверить?
- ✓ Не упустили ли мы что-то важное?

ТЕСТ ДИЗАЙН (ТД)

Тест-дизайн — набор техник, которые направленны на сокращение количества разрабатываемых и выполняемых тест-кейсов при сохранении достаточного тестового покрытия.

Цель техник тест дизайна — Определить тестовые условия, тестовые случаи (проверки) и тестовые данные.

Примеры:

Тестовый случай: проверить уникальность поля Login.

Тестовые условия: Приложение корректно установлено и доступно для тестирования.

Тестовые данные: в базе данных существует пользователь с Login=Ivan.

Основные техники тест-дизайна

Английское название	Русский эквивалент
Equivalence Class Testing	Классы эквивалентности
Boundary Value Testing	Граничные значения
State-Transition Testing	Диаграмма состояний и переходов
Decision Table Testing	Таблица решений
User Case Testing	Варианты использования
Allpairs Algorithm testing	Попарное тестирование
Orthogonal Arrays Testing	Ортогональная матрица
...	...

Классы эквивалентности

Два значения называются **эквивалентными**, когда приводят к одному результату. В таком случае неважно, взять значение А или Б для теста – так как они идентичны.

Эквивалентный класс — это одно или больше значений ввода, от которых ожидается сходное поведение, то есть они должны обрабатываться аналогичным образом. ..

А если реакция на все значения из одного класса эквивалетности одинаковая, то нет необходимости проверять все значения из этого класса.

Таким образом можно минимизировать количество тест-кейсов.

Пример 1:

В поле процент прибыли пользователь вводит число. Разрешенные значения находятся в диапазоне от 1 до 100 по требованию заказчика.

Шаг 1: Определите эквивалентные классы:

- Положительные числа (1–100)
- Отрицательные числа и ноль (<=0)
- Больше допустимого (>100)
- Буквы (a-z, a-Z, кириллица, арабский и т. д.)
- Специальные символы (!@# и т. д.)

Шаг 2: Выберите одно значение для каждого класса.

Итого у нас есть 5 тест-кейсов для проверки: 50, -10, 146, aBc, !

Пример 2:

Предположим, что книжный интернет магазин запускает новую кампанию "Больше тратишь — больше скидка".

Какие баги уже видны из этого условия ?

Потраченная сумма, руб-	**Скидка, %**
200 — 500	2
500—1000	3
1000 — 5000	4
5000 и более	5

Баг1: Непонятно, по какой ставке рассчитывается скидка, если потрачены следующие суммы: ровно 500 руб., ровно 1000 руб., ровно 5000 руб., так как каждая из этих сумм находится не в одной, а в двух корзинах со скидками.

Баг 2: Что означает "Потраченная сумма"? Это количество дензнаков, выплаченных только за книги, или полная сумма к оплате, включая оплату книг и расходы на доставку?

Баг 3: Для полноты картины нужно дописать эквивалентный класс от 0 до 199,99, на значения которого никакая скидка не распространяется.

Стоимость купленных книг, руб.	Скидка, %
0—199,99	0
200,00 — 499,99	2
500,00 — 999,99	3
1000,00 — 4999,99	4
5000,00 и более	5

Каждое значение внутри каждого класса является эквивалентным всем другим значениям этого класса. Потому что ко всем значениям класса должна применяться одинаковая логика кода.

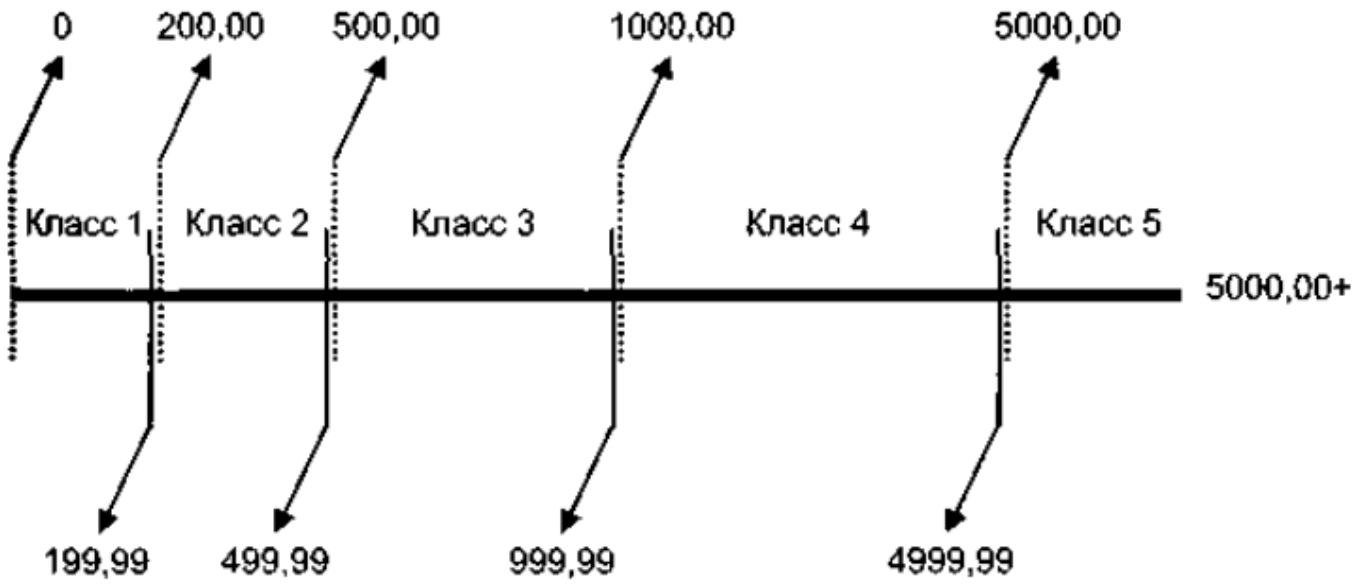
Класс 1:	0—199,99
Класс 2:	200,00 — 499,99
Класс 3:	500,00 — 999,99
Класс 4:	1000,00 — 4999,99
Класс 5:	5000,00 и более

Польза раскладывания значений ввода на эквивалентные классы состоит в том, что мы отсеиваем огромное количество значений ввода, использовать которые для тестирования просто бессмысленно. Отсев происходит путем применения знаний о тестировании **граничных значений**.

Граничные значения

Эта техника является "братом" разбиения на классы эквивалентности. Смысл этого подхода заключается в выборе значений на границах эквивалентных классов с минимальным шагом.

Вернемся к примеру книжного интернет-магазина



Поведение на границах эквивалентных областей имеет наибольшие шансы быть некорректным, таким образом **границы являются потенциальным источником дефектов.**

Для каждого эквивалентного класса может быть лишь один из трех вариантов:

- Есть только нижний предел (класс 5).
- Есть нижний и верхний пределы (класс 2, класс 3, класс 4).
- Есть только верхний предел (не рассматриваемый в данном примере класс, который ограничен только сверху отрицательным значением, непосредственно предшествующим классу 1).

Тестирование граничных значений

1. Тестируется нижний предел данного класса (если он имеется).
2. Затем тестируется верхний предел данного класса (если он имеется).
3. Затем тестируется любое значение внутри данного класса.
4. Затем тестируется верхний предел класса, непосредственно предшествующего данному классу (если предшествующий класс имеется).
5. Затем тестируется нижний предел класса, непосредствен но следующего за данным классом (если следующий класс имеется).

Можно записать граничные значения иначе

Min -1

Min

Average

Max

Max +1

Какие из этих тестов являются позитивными, а какие — негативными ?

Вернемся к примеру с книжным магазином

Потраченная сумма, руб-	**Скидка, %**
200 — 500	2
500—1000	3
1000 — 5000	4
5000 и более	5

Количество всех вариантов проверки для данного диапазона: 30 тыс (если учитывать копейки)

Используя методику тестирования эквивалентного класса, нам нужно лишь 5 вариантов данных:

1. 200,00;
2. 499,99;
3. 315,11;
4. 199,99;
5. 500,00.

Позитивные тесты

Итак, 5 вариантов данных для одного класса эквивалентности.

1. 200,00;
2. 499,99;
3. 315,11;
4. 199,99;
5. 500,00.

А сколько будет вариантов данных для 5 классов ?

Класс 1:	0—199,99
Класс 2:	200,00 — 499,99
Класс 3:	500,00 — 999,99
Класс 4:	1000,00 — 4999,99
Класс 5:	5000,00 и более

Класс	Значение	Ожидаемая ставка скидки, %
Класс 1	0	0
	100,00	
	199,99	
Класс 2	**200,00**	2
	315,11	
	499,99	
Класс 3	**500,00**	3
	659,23	
	999,99	
Класс 4	1000,00	4
	3265,26	
	4999,99	
Класс 5	5000,00	5
	5075,00	

Для тестирования одного класса эквивалентности понадобилось 5 тестов, а если тестировать 5 классов понадобиться **всего 14 тестов**, так как некоторые граничные условия будут учтены при тестировании других классов

Пример: **СИСТЕМА ПОДБОРА КРЕДИТА**

Возраст до 18 лет → Кредит не разрешен

Возраст от 18 до 65 лет →
- Потребительский кредит
- Кредит на недвижимость
- Кредит на покупку авто

Возраст старше 65 лет → Потребительский кредит

Пример: **СИСТЕМА ПОДБОРА КРЕДИТА**

Класс эквивалентности #1: от 1 до 17

Возраст до 18 лет → Кредит не разрешен

Класс эквивалентности #2: от 18 до 65

Возраст от 18 до 65 лет →

- Потребительский кредит
- Кредит на недвижимость
- Кредит на покупку авто

Класс эквивалентности #3: от 66 до 99

Возраст старше 65 лет → Потребительский кредит

Пример: **СИСТЕМА ПОДБОРА КРЕДИТА**

Класс эквивалентности #1: от 1 до 17

Возраст до 18 лет → Кредит не разрешен

Min -1:	пустое поле «Возраст»	> Сообщение об ошибке
Min:	**1**	> «Кредит не разрешен»
Average:	**11**	> «Кредит не разрешен»
Max:	**17**	> «Кредит не разрешен»
Max +1:	**18**	> «Потребительский кредит Кредит на недвижимость Кредит на покупку авто»

Пример: **СИСТЕМА ПОДБОРА КРЕДИТА**

Класс эквивалентности #2: от 18 до 65 лет

Возраст до 18 до 65 лет

- Потребительский кредит
- Кредит на недвижимость
- Кредит на покупку авто

Min -1:	17	> «Кредит не разрешен»
Min:	**18**	> 3 кредита
Average:	**40**	> 3 кредита
Max:	**65**	> 3 кредита
Max +1:	**66**	> «Потребительский кредит»

Пример: **СИСТЕМА ПОДБОРА КРЕДИТА**

Класс эквивалентности #3: от 66 до 99 лет

Возраст старше 65 лет

Потребительский кредит

Min -1:	65
Min:	**66**
Average:	**80**
Max:	**99**
Max +1:	**100**

> 3 кредита
> «Потребительский кредит»
> «Потребительский кредит»
> «Потребительский кредит»
> Система не позволяет ввести больше двух символов

Граничные значения для поля ДАТА РОЖДЕНИЯ

Формат: ДД.ММ.ГГГГ

День (значение):

Месяцы разбиты на 4 класса эквивалентности

	Январь, март, май, июль, август, октябрь, декабрь	Февраль (високосный год)	Февраль (не високосный год)	Апрель, июнь, сентябрь, ноябрь
Min – 1	00	00	00	00
Min	01	01	01	01
Average	15	15	15	15
Max	31	29	28	30
Max +1	32	30	29	31

День (формат):

Min -1: 1 символ **Min**: 2 символа **Average**: 2 символа **Max**: 2 символа **Max+1**: 3 символа

E-MAIL

Формат: name @ hostname.domain

	Local-part	@	Hostname	dot	domain
Min – 1	*Empty*	*Empty*	1 character	*Empty*	1 character
Min	1 character	@	2 characters	.	2 characters
Average	30 different characters	@	150 characters	.	6 characters
Max	64	@	243 characters	.	11 characters
Max +1	65	@@	244 characters	..	12 characters

E-MAIL

Формат: local-part @ hostname.domain
Size: 256 characters

ФОРМАТ

	Local-part	@	Hostname	dot	domain
Min – 1	*Empty*	*Empty*	1 character	*Empty*	1 character
Min	1 character	@	2 characters	.	2 characters
Average	30 characters	@	150 characters	.	6 characters
Max	64	@	243 characters	.	11 characters
Max +1	65	@@	244 characters	..	12 characters

РАЗМЕР

	Size	Examples
Min – 1	5 characters	*@a.by; aa.by; a@.by; a@aby; a@a.b*
Min	6 characters	a@a.by
Average	150 characters	…
Max	256 characters	…
Max +1	257 characters	…

Decision Table (таблица решений)

Decision Table (таблица решений) — техника, помогающая наглядно изобразить комбинаторику условий из ТЗ.

Таблица решений содержит триггерные условия, обычно комбинации значений «истина» и «ложь» для всех входных условий, и результирующие действия для каждой комбинации условий. **Каждый столбец таблицы соотносится с бизнес-правилом, определяющим уникальную комбинацию условий** и результат выполнения действий, связанных с этим правилом.

Таблицы решений чаще всего используют в бизнес-анализе для описания сложной бизнес-логики и в тестировании.

В тестировании таблица решений позволяет составить тест-кейсы и ничего не забыть при сложных комбинациях входных условий. Ведь каждая строка или столбец таблицы это готовый тест-кейс.

Как составить таблицу решений

Есть три упрощенных бизнес-правила.

- ✓ Если погода ясная и безветренная, и я не работаю, то я пойду за едой в магазин.
- ✓ Если погода дождливая или ветреная, то я закажу еду.
- ✓ Если я работаю, то я закажу еду.

Выделяем блоки правил

1. Если ясная и безветреная **погода**, и я не **работа**ю, то я схожу за едой в магазин.
2. Если дождливая или ветреная **погода**, то я закажу еду.
3. Если я **работа**ю, то я закажу еду.

Погода

Работа

Определяем входные и выходные параметры

1. Если **хорошая** погода, и я **не работаю**, то я **схожу за едой** в магазин
2. Если **плохая** погода, то я **закажу еду**
3. Если я **работаю**, то я **закажу еду**

№	Какая погода?	Я работаю?	Как я приобрету еду?
	Хорошая, Плохая	***Работаю, Не работаю***	***Схожу за едой, Закажу еду***

Перечисляем возможные комбинации входных параметров

№	Какая погода?	Я работаю?	Как я приобрету еду?
	Хорошая, Плохая	*Работаю, Не работаю*	*Схожу за едой, Закажу еду*
1	Хорошая	Работаю	Закажу еду
2	Хорошая	Не работаю	Схожу за едой
3	Плохая	Работаю	Закажу еду
4	Плохая	Не работаю	Закажу еду

СИСТЕМА ПОКУПКИ ПРОЕЗДНЫХ БИЛЕТОВ

ВИД ТРАНСПОРТА: АВТОБУС ● ТРОЛЛЕЙБУС ○ МЕТРО ○ ТРАМВАЙ ○

ПЕРИОД: ДЕНЬ ○ НЕДЕЛЯ ● ДЕКАДА ○ МЕСЯЦ ○

СПОСОБ ОПЛАТЫ: КАРТОЙ ● НАЛИЧНЫМИ ○

СУММА К ОПЛАТЕ: 10 РУБЛЕЙ

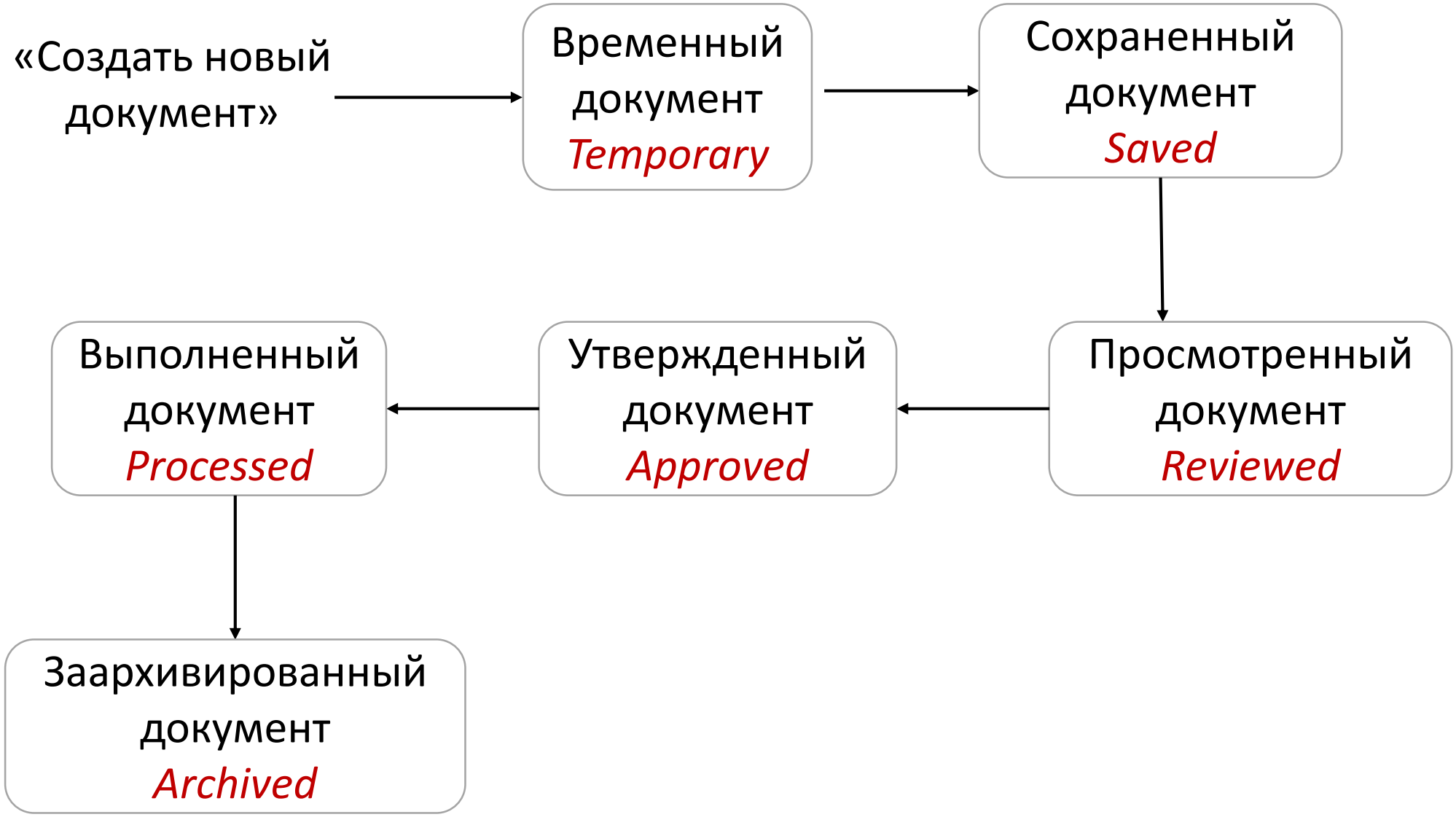
ВИД ТРАНСПОРТА	ПЕРИОД	ОПЛАТА	ОЖИДАЕМЫЙ РЕЗУЛЬТАТ (СУММА)
Автобус	День	Карта	5 рублей
Автобус	Неделя	Карта	10 рублей
Автобус	Декада	Наличными	13 рублей
Автобус	Месяц	Наличными	50 рублей
Троллейбус	День	Наличными	5 рублей
Троллейбус	Неделя	Карта	10 рублей
Троллейбус	Декада	Наличными	13 рублей
Троллейбус	Месяц	Карта	50 рублей
Метро	День	Наличными	5 рублей
Метро	Неделя	Карта	10 рублей
Метро	Декада	Карта	13 рублей
Метро	Месяц	Наличными	50 рублей
Трамвай	День	Карта	5 рублей
Трамвай	Неделя	Наличными	10 рублей
Трамвай	Декада	Наличными	13 рублей
Трамвай	Месяц	Карта	50 рублей

Тестирование ТАБЛИЦЫ СОСТОЯНИЙ

Система может показывать различные отклики в зависимости от текущих условий или предшествовавшей истории состояний. Данный метод позволяет тестировщику рассматривать систему с точки зрения её состояний, переходов между состояниями, входов или событий, активизирующих изменения состояний (переходы) и действия, к которым приводят эти переходы. Состояния системы или тестируемого объекта разделяемы, определяемы и конечны.

Таблица состояний демонстрирует связи между состояниями и входами и может подсказать возможные некорректные переходы.

СИСТЕМА ДОКУМЕНТООБОРОТА



Текущее состояние	Событие	Действие	Следующее состояние
null	«Создать новый документ»	Create	Temporary
null	«Создать новый документ»	Discard	null
Temporary	Заполнение документа	Save	Saved
Temporary	Заполнение документа	Cancel	null
Saved	Просмотр документа	Review	Reviewed
Saved	Просмотр документа	Reject	Rejected to creator
Reviewed	Утверждение документа	Approve	Approved
Reviewed	Утверждение документа	Reject	Archived
Approved	Выполнение действий по документу	Process	Processed
Processed	Архивация документа	Archive	Archived

СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

Сценарий использования (use case) описывает взаимодействия между участниками (включая пользователей и систему) приводящие к достижению конкретной цели.

Проще говоря, это то, что должен сделать пользователь, чтобы получить нужный результат.

Сценарий использования отвечает на вопросы:

- ✓ **кто** использует приложение или сайт;
- ✓ **что** хочет сделать пользователь, то есть каково конкретное действие;
- ✓ **цель** пользователя, которую он хочет достичь за счет выполнения задачи;
- ✓ **шаги**, которые необходимы, чтобы совершить нужное действие;
- ✓ описание того, **как программный продукт должен отреагировать** на действия пользователя.

> Сценарии использования описывают основной сценарий и все альтернативные сценарии, см. пример далее.

Пример реального сценария использования из спецификации ПО

Название	Подбор подходящего кредита
Описание	Сценарий описывает процесс подбора кредитных продуктов, подходящих под сумму и срок, введённые пользователем. Система подбирает доступные кредитные предложения и отображает их, отсортировав по возрастанию процентной ставки.
Триггер	Пользователь вводит параметры кредита и нажимает кнопку “Подобрать кредит”.
Актеры	Пользователь Система
Предусловия	1. Доступно стабильное интернет-соединение.
Постусловия	1. Система отображает список доступных кредитных продуктов, подходящих под указанные параметры. 2. Продукты отсортированы по возрастанию процентной ставки.

Бизнес-правила	1. Все введенные параметры (сумма, срок) должны пройти валидацию перед подбором кредитов.
	2. Подбор кредитных продуктов выполняется по правилам каждого продукта:
	  ○ диапазон суммы
	  ○ диапазон срока
	3. Система показывает только продукты, полностью удовлетворяющие условиям пользователя.
	4. Продукты сортируются по увеличению процентной ставки.
	5. После выбора продукта пользователь может перейти к оформлению кредитной заявки с предзаполненными полями.

Продолжение примера

Основной Сценарий	
Шаг	Действие
1	Пользователь вводит параметры кредита: сумму и срок (см. АС-01).
2	Система проверяет корректность введенных данных.
3	Система отправляет параметры в АБС для получения списка доступных кредитных продуктов.
4	Система фильтрует продукты (отображает только те, у которых диапазон суммы и срока включает введённые значения).
5	Система сортирует подобранные кредитные продукты по возрастанию процентной ставки.
6	Система отображает пользователю список подходящих кредитных продуктов.
7	Пользователь просматривает список и выбирает подходящий кредитный продукт.
8	Система перенаправляет пользователя на экран оформления кредитной заявки с предзаполненными значениями суммы и срока.

Продолжение примера

Альтернативный Сценарий #1 - Некорректные данные в заявке		
Заменяемый Шаг	Шаг	Действие
1	1а	Пользователь вводит некорректные данные.
	2а	Система отображает ошибку, информируя пользователя о необходимости исправить данные.
	3а	Пользователь исправляет ошибку.
	4а	Система проверяет исправленные данные и продолжает сценарий с шага 2 основного сценария.
Альтернативный Сценарий #2 - Изменение параметров кредита		
Заменяемый Шаг	Шаг	Действие
7	7б	Пользователь изменяет один из параметров.
	8б	Система обнуляет значения предварительных условий кредита и графика предстоящих платежей
	9б	Система выполняет Основной сценарий с шага 2

ПОПАРНОЕ ТЕСТИРОВАНИЕ

Попарное тестирование (Pairwise) — техника тест-дизайна, при которой тест-кейсы создаются таким образом, чтобы выполнить **все возможные отдельные комбинации каждой пары входных параметров.**

Тесты разрабатываются таким образом, что **для каждой пары** входных параметров существуют все возможные комбинации этих параметров.

Достаточно проверить каждую уникальную входную пару, так как ошибки чаще всего находятся на перекрестке двух параметров.

Пример из (https://habr.com/ru/companies/otus/articles/592575/

Рассмотрим на примере тестирования приложения для заказа автомобиля:

- ✓ С помощью приложения можно покупать и продавать машины. Приложение должно поддерживать оказание услуг в **Дели** и **Мумбаи**.
- ✓ В приложении должны содержаться регистрационные номера, которые могут быть **валидными** и **невалидным**. Оно должно разрешать продажу следующих марок автомобилей: BMW, Audi и Mercedes.
- ✓ Доступны два типа бронирования: бронирование через интернет и в офлайн-магазине.
- ✓ Заказы доступны к размещению только в рабочие часы.

Шаг 1. Перечислим задействованные переменные.

Категория заказа

а. Купить
б. Продать

Местоположение

а. Дели
б. Мумбаи

Марка автомобиля

а. BMW
б. Audi
в. Mercedes

Регистрационные номера

а. Валидные (5000)
б. Не валидные

Тип заказа

а. Заказ через Интернет
б. Заказ в магазине

Время заказа

а. Рабочие часы
б. Нерабочие часы

Если тестировать все возможные допустимые комбинации: 2 x 2 x 3 x 5000 x 2 x 2 = получаем 240 тысяч комбинаций ! Кроме того, недопустимых комбинаций вообще может быть бесконечное количество.

Шаг №2: Упростим.

Сокращаем регистрационные номера до двух типов:

- Валидный регистрационный номер
- Не валидный регистрационный номер

Категория заказа

а. Купить
б. Продать

Местоположение

а. Дели
б. Мумбаи

Марка автомобиля

а. BMW
б. Audi
в. Mercedes

Регистрационные номера

а. Валидные ~~(5000)~~
б. Не валидные

Тип заказа

а. Заказ через Интернет
б. Заказ в магазине

Время заказа

а. Рабочие часы
б. Нерабочие часы

Теперь посчитаем количество возможных комбинаций: = 2 x 2 x 3 x 2 x 2 x 2 = 96

Шаг 3. Упорядочивание задействованных переменных и значений.

Категория заказа	Местоположение	Марка авто	Регистрационный номер	Тип заказа	Время заказа
Покупка	Дели	BMW	Валидные	через интернет	Рабочие часы
Продажа	Мумбаи	Audi	Невалидные	в магазине	Нерабочие часы
		Mercedes			

Теперь **отсортируем переменные** так, чтобы переменные с наибольшим количеством значений шли первыми, а с наименьшим — последними.

Марка авто	Категория заказа	Местоположение	Регистрационный номер	Тип заказа	Время заказа
3	2	2	2	2	2

Шаг 4. Расставляем переменные для создания набора тестов.

Марка авто	Категория заказа	Ме
BMW		
BMW		
Audi		
Audi		
Mercedes		
Mercedes		

1

Марка авто	Категория заказа	Мест
BMW	Покупка	
BMW	Продажа	
Audi	Покупка	
Audi	Продажа	
Mercedes	Покупка	
Mercedes	Продажа	

2

Шаг 4. Расставляем переменные для создания набора тестов.

Марка авто	Категория заказа	Местоположен
BMW	Покупка	Дели
BMW	Продажа	Мумбаи
Audi	Покупка	Дели
Audi	Продажа	Мумбаи
Mercedes	Покупка	Дели
Mercedes	Продажа	Мумбаи

3

Марка авто	Категория заказа	Местоположен
BMW	Покупка	Дели
BMW	Продажа	Мумбаи
Audi	Покупка	Мумбаи
Audi	Продажа	Дели
Mercedes	Покупка	Дели
Mercedes	Продажа	Мумбаи

4

Теперь у нас есть Покупка&Дели, но нет Покупка&Мумбаи. Есть Продажа&Мумбаи, но нет Продажа&Дели. Поменяем значения из второго набора в третьем столбце.

Повторим шаги для столбцов 3 и 4.

Марка авто	Категория заказа	Местоположение	Регистрационный номер	Т
BMW	Покупка	Дели	Валидный	
BMW	Продажа	Мумбаи	Невалидный	
Audi	Покупка	Мумбаи	Валидный	
Audi	Продажа	Дели	Невалидный	
Mercedes	Покупка	Дели	Валидный	
Mercedes	Продажа	Мумбаи	Невалидный	

Аналогично, если сравнить столбцы 3 и 4, каждое значение из столбца 3 имеет пару с обоими значениями из столбца 4. Но если сравнить второй и четвертый столбец, у нас есть комбинации Покупка&Валидный и Продажа&Невалидный, но нет комбинаций Покупка&Невалидный и Продажа&Валидный. Следовательно, нам надо поменять местами последний набор значений в четвертом столбце.

Марка авто	Категория заказа	Местоположение	Регистрационный номер	Тип заказа	Время заказа
BMW	Покупка	Дели	Валидный	Через интернет	Рабочие часы
BMW	Продажа	Мумбаи	Невалидный	В магазине	Нерабочие часы
Audi	Покупка	Мумбаи	Валидный	В магазине	Рабочие часы
Audi	Продажа	Дели	Невалидный	Через интернет	Нерабочие часы
Mercedes	Покупка	Дели	Невалидный	В магазине	Рабочие часы
Mercedes	Продажа	Мумбаи	Валидный	Через интернет	Нерабочие часы

С шестым столбцом (Время заказа) у нас проблемка: не хватает пар Покупка&Нерабочие часы и Продажа&Рабочие часы. Нам не удастся получить недостающие пары, поменяв значения местами, поскольку мы ранее уже поменяли местами все строки, и если мы снова начнём их менять, то есть риск пропустить другие возможные пары. Поэтому **добавим еще два тестовых случая, которые содержат эти пары**.

Марка авто	Категория заказа	Местоположение	Регистрационный номер	Тип заказа	Время заказа
BMW	Покупка	Дели	Валидный	Через интернет	Рабочие часы
BMW	Продажа	Мумбаи	Невалидный	В магазине	Нерабочие часы
	Покупка				Нерабочие часы
Audi	Покупка	Мумбаи	Валидный	В магазине	Рабочие часы
Audi	Продажа	Дели	Невалидный	Через интернет	Нерабочие часы
	Продажа				Рабочие часы
Mercedes	Покупка	Дели	Невалидный	В магазине	Рабочие часы
Mercedes	Продажа	Мумбаи	Валидный	Через интернет	Нерабочие часы

Марка авто	Категория заказа	Местоположение	Регистрационный номер	Тип заказа	Время заказа
BMW	Покупка	Дели	Валидный	Через интернет	Рабочие часы
BMW	Продажа	Мумбаи	Невалидный	В магазине	Нерабочие часы
	Покупка				Нерабочие часы
Audi	Покупка	Мумбаи	Валидный	В магазине	Рабочие часы
Audi	Продажа	Дели	Невалидный	Через интернет	Нерабочие часы
	Продажа				Рабочие часы
Mercedes	Покупка	Дели	Невалидный	В магазине	Рабочие часы
Mercedes	Продажа	Мумбаи	Валидный	Через интернет	Нерабочие часы

Заполним пустые строки!

Заполним пустые ячейки на свое усмотрение, потому что другие значения переменных являются произвольными (обозначим знаком тильды ~)

Марка авто	Категория заказа	Местоположение	Регистрационный номер	Тип заказа	Время заказа
BMW	Покупка	Дели	Валидный	Через интернет	Рабочие часы
BMW	Продажа	Мумбаи	Невалидный	В магазине	Нерабочие часы
~BMW	Покупка	~Дели	~Валидный	~Через интернет	Нерабочие часы
Audi	Покупка	Мумбаи	Валидный	В магазине	Рабочие часы
Audi	Продажа	Дели	Невалидный	Через интернет	Нерабочие часы
~Audi	Продажа	~Мумбаи	~Невалидный	~В магазине	Рабочие часы
Mercedes	Покупка	Дели	Невалидный	В магазине	Рабочие часы
Mercedes	Продажа	Мумбаи	Валидный	Через интернет	Нерабочие часы

Итого мы получили всего 8 тест-кейсов вместо 96.

Этот пример показывает, насколько эффективной может быть **техника попарного тестирования**. Она здорово повышает шансы найти баги, при этом сохранив время.

Для генерации комбинаций тестов методом попарного тестирования можно использовать онлайн-инструменты

Техника попарного тестирования помогает существенно уменьшить количество комбинаций проверок, **достаточных для обеспечения необходимого уровня качества** программного обеспечения.

По статистике 87-93% ошибок возникает при встрече двух параметров. При большом сокращении количества тестов и времени на них можно пожертвовать 7 % допустимых ошибок).

Однако эта техника имеет и некоторые ограничения. Она не сработает, если:

- ✓ выбранные для тестирования значения некорректны;
- ✓ мало внимания уделяется комбинациям, которые могут привести к ошибке с высокой долей вероятности;
- ✓ взаимодействие между переменными недостаточно изучено.

ПОПАРНОЕ ТЕСТИРОВАНИЕ

#	Тип подключения	Микрофон	Подсветка
1	Проводной	Включен	Включена
2	Проводной	Включен	Выключена
3	Проводной	Выключен	Включена
4	Проводной	Выключен	Выключена
5	Беспроводной	Включен	Включена
6	Беспроводной	Включен	Выключена
7	Беспроводной	Выключен	Включена
8	Беспроводной	Выключен	Выключена

Пример 2. Возьмем наушники с характеристиками:

- Тип подключения (беспроводной или проводной)
- Микрофон (включен или выключен)
- Подсветка (включена или выключена)

Для данного примера число тестов сократилось вдвое!

#	Тип подключения	Микрофон	Подсветка
1	Беспроводной	Выключен	Выключена
2	Проводной	Включен	Выключена
3	Беспроводной	Включен	Включена
4	Проводной	Выключен	Включена

ТЕСТ ДИЗАЙН НА ОСНОВЕ ОПЫТА

Подходы основаны на предыдущем опыте использования аналогичных приложений / платформ.

- ✓ Предположения об ошибках (error guessing technique);
- ✓ Исследовательское тестирование (exploratory testing).

Вопрос:

Рассмотренные методы относятся к методам Черного ящика или Белого ящика ?

- ✓ Классы эквивалентности
- ✓ Граничные значения
- ✓ Таблица решений
- ✓ Диаграмма состояний
- ✓ Сценарии использования
- ✓ Попарное тестирование

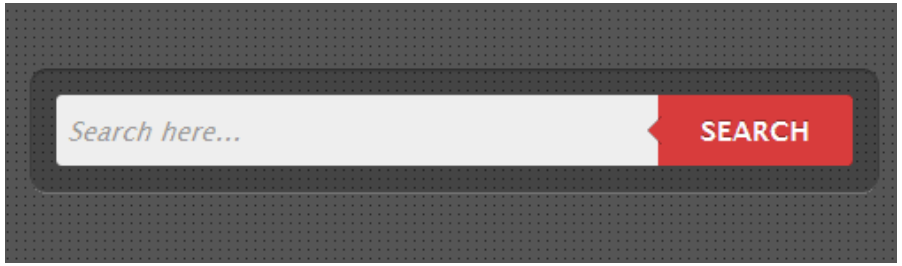
ПОЗИТИВНОЕ ТЕСТИРОВАНИЕ

- ✓ Ввод корректного номера телефона с пробелами;
- ✓ Ввод корректного номера телефона с дефисами;
- ✓ Ввод корректного номера телефона со скобками;
- ✓ Ввод номера разных операторов;
- ✓ Выбор другой страны (должен измениться код страны)
- ✓ Смена IP адреса (должна измениться страна по умолчанию).

НЕГАТИВНОЕ ТЕСТИРОВАНИЕ

- ✓ Оставить поля пустыми;
- ✓ Ввести спец символы или буквы;
- ✓ Ввести телефон за пределами минимума и максимума;
- ✓ Написать телефон, потом сменить страну;
- ✓ Набрать номер состоящий из нулей.

ПОЗИТИВНОЕ ТЕСТИРОВАНИЕ



НЕГАТИВНОЕ ТЕСТИРОВАНИЕ

- ✓ Проверить, что placeholder исчезает, когда фокус находится в поле;
- ✓ Поиск по нажатию на кнопку Search;
- ✓ Поиск по нажатию “Enter”;
- ✓ Поиск существующих страниц;
- ✓ Поиск с применением операторов;
- ✓ Вставка текста поиска из буфера обмена;
- ✓ Поиск по нескольким словам.

- ✓ Оставить поле пустым и нажать Search;
- ✓ Поиск по спец символам;
- ✓ Текст поиска за пределами минимума и максимума;
- ✓ Поиск страниц, которых не существует.